9.2 Modeling your schema first

Before the ACME Corp identity team wrote a single policy, they developed a schema.

A Cedar *schema* defines the shape of entities, actions, and contexts that your policies can reference. It functions like a contract between your authorization logic and the rest of your system: the schema specifies what's permitted, and Cedar's compiler ensures these rules are followed. If a policy tries to use an attribute, action, or entity type not included in the schema, it won't compile.

Without a schema, policies risk becoming inconsistent and prone to errors. With a schema, you gain:

- **Type safety**—catch mistakes before deployment.
- **Precise vocabulary**—everyone uses consistent names for actions, entity types, and attributes.
- **Tooling support**—IDEs and CI/CD pipelines validate policy changes.

These benefits make the schema more than just a formality; it becomes the backbone that keeps your Cedar policies reliable and consistent as the system evolves.

9.2.1 ACME's core entities and actions

ACME's Customer Collaboration platform covers a few key concepts:

- **Employees**—internal staff with assigned roles, departments, and managers.
- **Customers**—external individuals with limited access.
- **Documents**—shared files with owners, classifications, and a delegatable flag.
- **Teams**—groups of employees assigned to projects or departments, or customers assigned to specific clients.